

Mini-progetto: *Creature Loader (HTTP · JSON · OOP)*

Obiettivo

Realizzare un programma Python che:

1. recupera dati da un servizio HTTP pubblico,
2. gestisce correttamente gli errori,
3. salva i dati su file JSON (serializzazione),
4. legge i dati da file JSON (deserializzazione),
5. ricostruisce un oggetto Python **passando i dati direttamente al costruttore**.

Il progetto deve dimostrare l'uso corretto di:

- modulo `requests`
 - `try / except`
 - JSON (`json.dump`, `json.load`)
 - costruttori di classe con `**kwargs`
 - separazione delle responsabilità
-

◆ Servizio HTTP

Utilizzare la **PokeAPI**:

- `https://pokeapi.co/api/v2/pokemon/{name_or_id}`

Esempi validi:

- `pikachu`
- `25`

La risposta è in formato JSON.

♦ Flusso logico richiesto

- HTTP request
- ↓
- lettura dati (Reader)
- ↓
- normalizzazione in dizionario
- ↓
- salvataggio su file JSON
- ↓
- lettura file JSON
- ↓
- creazione oggetto Creature tramite Creature(**dict)



Classi richieste

1 Classe Creature

Rappresenta il modello dati.

Attributi minimi:

- `id` (int)
- `name` (str)
- `height` (int)
- `weight` (int)
- `types` (list[str])
- `stats` (dict[str, int])

Costruttore:

- `Creature(**data_dict)`

Dove `data_dict` contiene **esattamente** le chiavi corrispondenti agli attributi.

Metodi richiesti:

- `to_dict(self) -> dict`
→ restituisce un dizionario serializzabile
 - un metodo di stampa o riepilogo (`summary`, `__str__`, ecc.)
-

2 Classe Reader (es. `PokemonReader`)

Responsabilità:

- effettuare la richiesta HTTP con `requests`
- gestire errori di rete e HTTP con costrutti `try except`
- estrarre dal JSON della API **solo i dati necessari**
- restituire **un dizionario pronto per essere passato a `Creature(**dict)`**

Esempio concettuale:

- `data = reader.fetch_as_dict("pikachu")`
 - `creature = Creature(**data)`
-

3 JSONReader & JSONWriter

Implementare delle classi dedicate che:

- salvino un dizionario su file JSON
- leggano un file JSON e restituisca un dizionario

Percorso di salvataggio:

- `data/creatures/<nome>.json`

Le cartelle devono essere create automaticamente se non esistono.



Gestione delle eccezioni (obbligatoria)

Devono essere gestiti con `try` / `except` **almeno** i seguenti casi:

HTTP / rete

- timeout
- errore di connessione
- errore HTTP (es. risorsa non trovata)

File / JSON

- file non trovato
- JSON malformato
- dati non compatibili con il costruttore di `Creature`

Il programma **non deve andare in crash**:
mostrare messaggi chiari all'utente.



Programma principale (**main**)

Il programma deve:

1. chiedere all'utente un nome o id
2. recuperare i dati dalla API
3. salvarli su file JSON
4. creare un oggetto `Creature` tramite `Creature(**dict)`
5. stampare un riepilogo
6. chiedere se ricaricare la creatura dal file
7. se sì:
 - leggere il file JSON
 - ricreare l'oggetto **sempre tramite** `Creature(**dict)`
 - ristampare il riepilogo

Consegna richiesta

- file Python (`.py`)
- cartella `data/creatures/` con **almeno un file JSON**
- codice leggibile e correttamente strutturato

Note

- Non è richiesta interfaccia grafica
- Non sono richieste librerie esterne oltre `requests`
- Il focus è sulla **logica e sull'architettura**, non sulla quantità di codice